

Tecnológico de Monterrey - Campus Monterrey
Programación de estructuras de datos y algoritmos fundamentales

Evidencia de Pruebas: Evidencia 1: Actividad Integral de Conceptos Básicos y Algoritmos Fundamentales

Juan Ángel Narváez Torres - A00843809

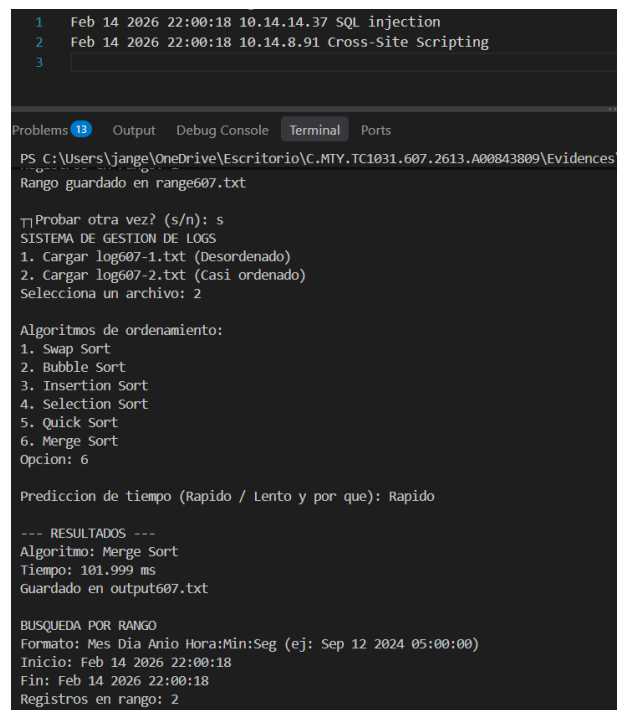
Septiembre 20, 2026
Monterrey, Nuevo León

A. Plan de Pruebas

El objetivo de las pruebas fue validar que el programa lea adecuadamente los datos de entrada, ejecute los algoritmos de ordenamiento correctamente y garantice que la búsqueda por rango retorne la totalidad de los registros correspondientes, incluyendo casos con fechas duplicadas.

B. Casos de Prueba Ejecutados

1. **Caso Límite - Fechas duplicadas:** Búsqueda utilizando una misma fecha de inicio y fin dentro del archivo log607-2.txt para verificar la recuperación completa de todos los registros coincidentes.



```
1 Feb 14 2026 22:00:18 10.14.14.37 SQL injection
2 Feb 14 2026 22:00:18 10.14.8.91 Cross-Site Scripting
3

Problems 13 Output Debug Console Terminal Ports
PS C:\Users\jange\OneDrive\Escritorio\C.MTY.TC1031.607.2613.A00843809\Evidences\F
Rango guardado en range607.txt

Probar otra vez? (s/n): s
SISTEMA DE GESTION DE LOGS
1. Cargar log607-1.txt (Desordenado)
2. Cargar log607-2.txt (Casi ordenado)
Selecciona un archivo: 2

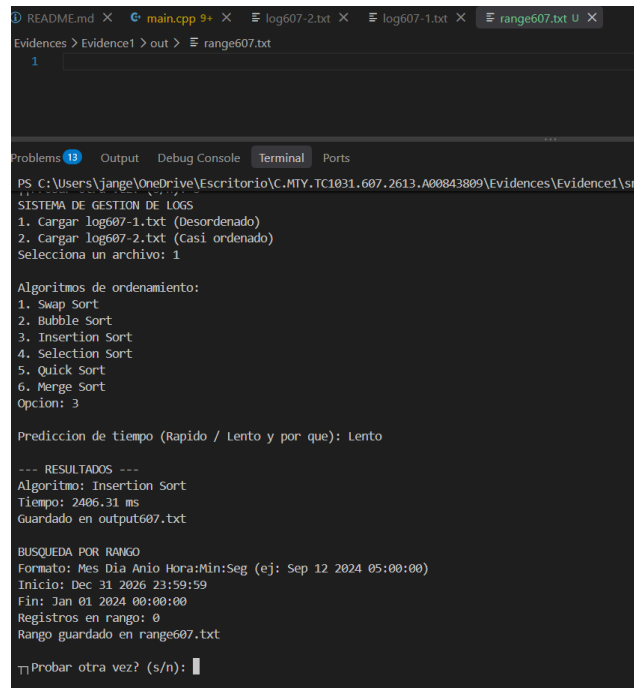
Algoritmos de ordenamiento:
1. Swap Sort
2. Bubble Sort
3. Insertion Sort
4. Selection Sort
5. Quick Sort
6. Merge Sort
Opcion: 6

Prediccion de tiempo (Rapido / Lento y por que): Rapido

--- RESULTADOS ---
Algoritmo: Merge Sort
Tiempo: 101.999 ms
Guardado en output607.txt

BUSQUEDA POR RANGO
Formato: Mes Dia Añio Hora:Min:Seg (ej: Sep 12 2024 05:00:00)
Inicio: Feb 14 2026 22:00:18
Fin: Feb 14 2026 22:00:18
Registros en rango: 2
```

2. **Rango Vacío:** Ingreso de un rango donde la fecha inicial es posterior a la fecha final (ej. Inicio: Dec 31 2026, Fin: Jan 01 2026).



```
PS C:\Users\jange\OneDrive\Escritorio\C.MTY.TC1031.607.2613.A00843809\Evidences\Evidence1\src>
SISTEMA DE GESTION DE LOGS
1. Cargar log607-1.txt (Desordenado)
2. Cargar log607-2.txt (Casi ordenado)
selecciona un archivo: 1

Algoritmos de ordenamiento:
1. Swap Sort
2. Bubble Sort
3. Insertion Sort
4. Selection Sort
5. Quick Sort
6. Merge Sort
Opcion: 3

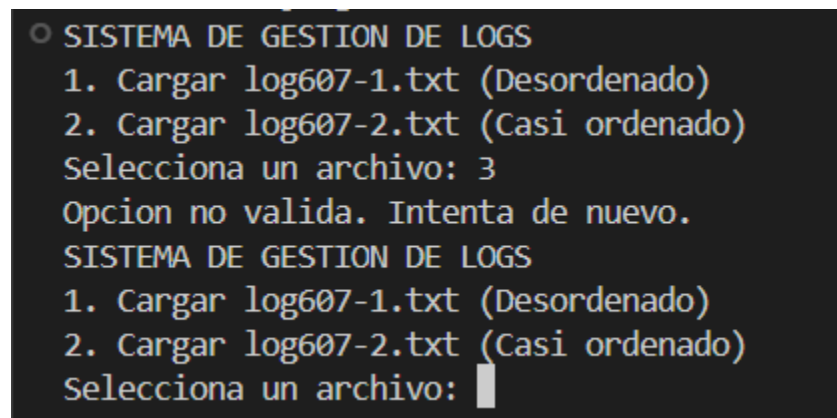
Prediccion de tiempo (Rapido / Lento y por que): Lento

--- RESULTADOS ---
Algoritmo: Insertion Sort
Tiempo: 2406.31 ms
Guardado en output607.txt

BUSQUEDA POR RANGO
Formato: Mes Dia Ario Hora:Min:Seg (ej: Sep 12 2024 05:00:00)
Inicio: Dec 31 2026 23:59:59
Fin: Jan 01 2024 00:00:00
Registros en rango: 0
Rango guardado en range607.txt

Probar otra vez? (s/n):
```

3. **Manejo de Errores - Archivo inexistente:** Validación del comportamiento del programa al intentar seleccionar un número equivocado de archivo, verificando que finalice de manera segura desplegando la advertencia correspondiente.



```
SISTEMA DE GESTION DE LOGS
1. Cargar log607-1.txt (Desordenado)
2. Cargar log607-2.txt (Casi ordenado)
Selecciona un archivo: 3
Opcion no valida. Intenta de nuevo.
SISTEMA DE GESTION DE LOGS
1. Cargar log607-1.txt (Desordenado)
2. Cargar log607-2.txt (Casi ordenado)
Selecciona un archivo:
```

C. Resultados y Tiempos de Ejecución

A continuación se detallan las mediciones de tiempo de ejecución obtenidas para cada algoritmo:

Algoritmo	log607-1.txt	log607-2.txt
Swap Sort	10657.8 ms	192.597 ms
Bubble Sort	10752.1 ms	19.177 ms

Algoritmo	log607-1.txt	log607-2.txt
Insertion Sort	2406.31 ms	9.932 ms
Selection Sort	177.528 ms	171.649 ms
Quick Sort	50.475 ms	48.374 ms
Merge Sort	104.281 ms	101.999 ms

D. Conclusión

Al analizar los tiempos de ejecución registrados, se evidencia que no existe un algoritmo de ordenamiento óptimo para todos los escenarios. Insertion Sort demuestra el rendimiento superior cuando los datos ya cuentan con un orden parcial; sin embargo, en conjuntos de datos desordenados o aleatorios, algoritmos eficientes como Merge Sort o Quick Sort ofrecen mejores resultados.

Asimismo, la implementación de búsquedas lineales directas resulta insuficiente en aplicaciones reales, haciendo indispensable el control adecuado de límites inferiores y superiores para mitigar colisiones por registros con marcas temporales idénticas.